

UNIVERSITI TEKNOLOGI MARA

COLLEGE OF COMPUTING, INFORMATICS, AND MATHEMATICS

BACHELOR OF INFORMATION SYSTEMS (HONS.) INTELLIGENT SYSTEM ENGINEERING
CDCS259

OPTIMIZATION ALGORITHMS AND APPLICATION (ISP611)

INDIVIDUAL ASSIGNMENT

TITLE:

PERSONALISED NUTRI-TECH OPTIMIZATION FOR WOMEN'S HEALTH

PREPARED BY:

RAIHANUL 'IZZAH HUDA BINTI KHAZALI (2025306745)

GROUP:

KCDCS2595B

PREPARED FOR : DR. TANIZA BINTI TAJUDDIN

TABLE OF CONTENTS

1. INTRODUCTION 3
2. CASE STUDY IDENTIFICATION 4
3. OBJECTIVE FUNCTION 5
    3.1 OBJECTIVES DEFINITION 5
    3.2 FITNESS FUNCTIONS 5
    3.3 COMBINED OBJECTIVE FORMULATION 6
4. CHROMOSOME REPRESENTATION 7
5. PROGRAMMING LANGUAGE 9
6. STEPS IN GA AND CODING IMPLEMENTATION 9
7. RESULTS AND DISCUSSION 13
8. SUGGESTION FOR IMPROVEMENT 17
9. CONCLUSIONS 18
10. REFERENCES 19
11. APPENDICES 20

1. INTRODUCTION

The rapid evolution of data-driven health technology has catalyzed a shift from generalized dietary guidelines toward highly personalized nutrition interventions. This paradigm shift recognizes that optimal health is not a "one-size-fits-all" concept but rather a precise balance tailored to an individual's metabolic profile, lifestyle, and genetic predispositions. Advanced computational models now achieve accuracy rates exceeding 90% in predicting how individuals respond to specific nutrients, driven by breakthroughs in Artificial Intelligence and a deeper understanding of the complex interplay between environmental and biological factors. For women, these personalized requirements are particularly vital across various life milestones ranging from prenatal support in the early 20s to bone density management in the late 60s, requiring a sophisticated approach to supplement selection that ensures both safety and efficacy.

To address the logistical complexity of selecting the ideal combination from an expansive inventory of supplements, the Genetic Algorithm (GA) emerges as a superior optimization tool. Manual selection from over 50 supplement variations is not only inefficient but also prone to overlooking critical nutrient-gene interactions that generic formulations ignore. By utilizing probabilistic transition rules such as selection, crossover, and mutation, the GA can navigate non-linear variables to identify a "Nutrient Stack" that meets 100% of a user's biological needs. This automated approach successfully balances the technical "Trade-off Conflict" between absolute nutritional accuracy, user convenience—managed by minimizing "pill fatigue"—and economic affordability, ensuring a competitive and effective personalized health solution.

2. CASE STUDY IDENTIFICATION

The selected case study is the Optimization of Personalized Nutri-Tech Subscriptions for Women. This problem addresses the growing demand for data-driven health technology, where the objective is to select the perfect combination of vitamins and minerals from a large inventory to meet a woman's Recommended Daily Intake (RDI) for milestones like prenatal care, bone density, or hormonal health among women in their early 20s up to women in their late 60s.

Personalized nutrition has moved from generic healthy eating guidelines toward interventions tailored to an individual's unique characteristics, which include metabolic profile, genetics, and lifestyle. The advanced computational models have demonstrated accuracy rates exceeding 90% in predicting responses to dietary interventions (Singar et al., 2024). This pattern shift is caused by advancements in Artificial Intelligence (AI) and the recognition that unique nutritional requirements are created by a complex interplay of various environmental and genetic factors (Nourazarian & Vaziri, 2026). Recent patent reviews further highlight that genome-based personalized nutrition is a rapidly expanding field of technology aimed at precise health management (Park et al., 2024).

Using a Genetic Algorithm (GA) is essential here because manual selection from 50+ supplement variations is inefficient. GAs are highly favored for such complex optimization issues because they can handle both stationary and non-stationary objective functions, as well as linear and non-linear variables (Lam et al., 2024). Furthermore, genotype-based nutritional supplementation has proven effective in addressing specific nutrient-gene interactions that generic supplements ignore (Wang et al., 2022). By adopting probabilistic transition rules including selection, crossover, and mutation, the GA identifies a "Nutrient Stack" that hits 100% of biological needs while minimizing "pill fatigue" and monthly costs (Lam et al., 2024).

3. OBJECTIVE FUNCTION

3.1 OBJECTIVES DEFINITION

There are three objectives identified in this case study which involved from nutritional integrity, user compliance and market competitiveness.

1. Nutritional Integrity

   To ensure the specific needs for pregnancy (e.g., Folic Acid) or bone health (e.g., Calcium) are met without dangerous over-supplementation.

2. User Compliance

   To reduce the physical burden of consumption to ensure the user actually takes the supplements.

3. Market Competitiveness

   To maintain a subscription price that is lower than buying individual bottles separately.

3.2 FITNESS FUNCTIONS

1. Nutritional Accuracy Fitness

   This measures the proximity to the Target RDI. It uses a "soft limit" approach where meeting the goal is rewarded, but exceeding it (toxicity risk) or falling short is penalized.

$$f_A = \sum_{i=1}^{n} \left(\frac{\text{Provided Nutrient}_i}{\text{Target RDI}_i} \right)$$

Note: Values are capped at 1.0 per nutrient to prevent a surplus in one area from masking a deficiency in another.

2. Pill Count Penalty

   This is a linear penalty function designed to combat "pill fatigue."

   As the number of pills increases, the overall fitness score drops, forcing the GA to favor high-potency multivitamins over multiple single-ingredient pills.

$$f_P = \text{Total Pills in Chromosome} \times \text{Penalty Factor}$$

3. Economic Cost Fitness

   This function evaluates the total RM (Ringgit Malaysia) value of the selected items.

$$f_C = \sum(\text{Cost of Selected Supplement}_j)$$

   This ensures the algorithm identifies the most cost-effective "stack" from the warehouse inventory.

3.3 COMBINED OBJECTIVE FORMULATION

The fitness of a chromosome x is calculated as:

$$f(x) = (w_1 \cdot A) - (w_2 \cdot P) - (w_3 \cdot C)$$

Where:

- $w1$, $w2$, $w3$: Weights assigned to each objective (must sum to 1.0).
- A: Normalized Accuracy score.
- P: Total number of pills.
- C: Total monetary cost.

4. CHROMOSOME REPRESENTATION

In this model, each potential solution which is the chromosome is represented as a bit-string where each bit which is a gene signifying the presence or absence of a specific item.

The following Table 4.0 (a) specifies the attributes for each individual supplement item available in the inventory:

Supplement	Iron (mg)	Folic Acid (mcg)	Calcium (mg)	Cost (RM)
Prenatal Basic	18	400	0	1.50
Calcium + Folic	0	200	500	2.00
High Iron	27	0	0	1.20
High Folic	0	600	0	0.80
Daily Balance	9	100	250	1.10

Table 4.0 (a): Attributes for each supplement item available in inventory

From the table above, each supplement is mapped to a specific Gene Index. The chromosome's binary state (0 or 1) determines whether these nutritional attributes are added to the total stack performance.

The state of existence for each item is represented via binary numbers, which directly impacts the three core strategic objectives of the Nutri-Tech system:

Nutritional Integrity (A): Selected genes must sum to meet the user's RDI without dangerous over-supplementation.

User Compliance (P): The total number of "1" bits is monitored to reduce "pill fatigue".

Market Competitiveness (C): The sum of costs from active genes must remain affordable.

The Table 4.0 (b) below illustrates how different chromosomes represent distinct potential solutions for a user:

Gene Index	**0 (Prenatal)**	**1 (Calcium+)**	**2 (Iron)**	**3 (Folic)**	**4 (Daily)**	**Interpretation**
Chromosome 1	1	0	1	0	0	**High Iron + Prenatal:** Prioritizes iron-rich needs.
Chromosome 2	0	1	0	1	0	**Calcium + High Folic:** Targets bone health and folic acid.

Table 4.0 (b): Different chromosome representation for a distinct possible solution

Each chromosome is evaluated using a Multi-Objective Maximization function. The fitness score $f(x)$ is not just a simple sum.

$$f(x) = (w_1 \cdot A) - (w_2 \cdot P) - (w_3 \cdot C)$$

It is a weighted balance designed to resolve the "Trade-off Conflict" between medical precision and consumer convenience:

1. Accuracy (A): Calculated by comparing the sum of nutrients from all "1" genes against the Target RDI. To ensure Nutritional Integrity, values are capped at 1.0 per nutrient to prevent over-supplementation from inflating the score.
2. Pill Penalty (P): A linear penalty applied based on the total count of active genes (pills). This ensures User Compliance by favoring high-potency combined supplements over a large stack of single-ingredient pills.
3. Cost Fitness (C): The total RM value of selected supplements. This maintains Market Competitiveness by identifying the most cost-effective combination.

By adjusting the weights (w), the system can be tuned to different service tiers, such as prioritizing absolute medical accuracy for high-risk prenatal care or prioritizing low cost for budget-conscious users

5. PROGRAMMING LANGUAGE

Python is used for this application. It is the industry standard for optimization due to NumPy for vector-based fitness calculations and Matplotlib for visualizing convergence.

6. STEPS IN GA AND CODING IMPLEMENTATION

Steps in GA (Details)	**Coding Implementation (Details)**
1. Initialization	Captures user constraints (Y/N) and uses np.random.randint to create a binary population.

	`def run_ga(mutation_rate, pop_size, budget, user_constraints, generations=50):`
	
	`    population = np.random.randint(2, size=(pop_size, GENES))`
	
	`    history = []`

2. Evaluation	Select the best combination of dietary supplements to meet nutrient targets (Iron, Folic Acid, Calcium) while staying within a budget.
	
	`scores = np.array([calculate_fitness(ind, budget, no_constraints) for ind in population])`
	
	`        history.append(np.max(scores))`

3. Selection	Implements Roulette Wheel Selection by normalizing fitness scores into a probability distribution.
	
	`        if np.sum(scores) < 1e-6:`
	
	`            population = np.random.randint(2, size=(pop_size, len(SUPPLEMENT_INVENTORY)))`
	
	`            continue`

	`parents = population[np.random.choice(pop_size, size=pop_size, p=scores/np.sum(scores))]`

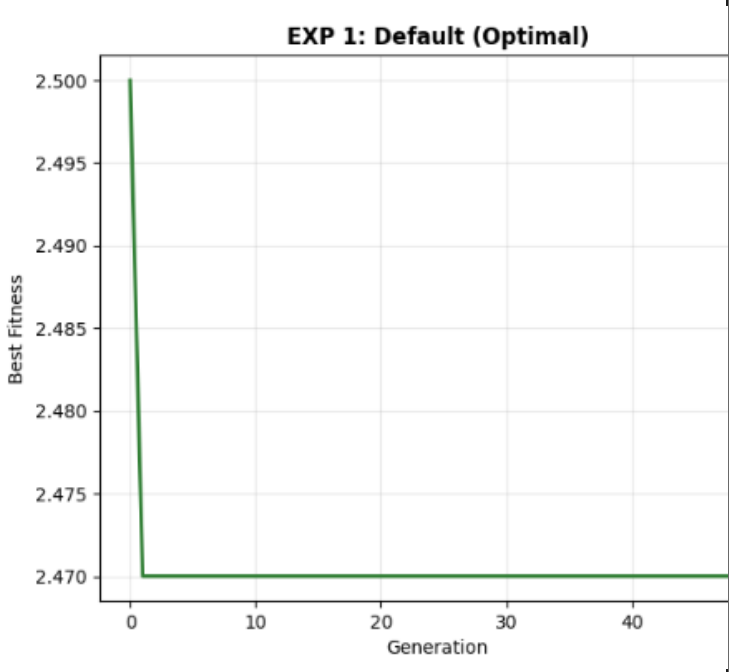
4. Crossover	Uses Single-Point Crossover to swap genetic traits between high-performing parent stacks.
	
	`offspring = []`
	
	`for i in range(0, pop_size, 2):`
	
	`p1, p2 = parents[i], parents[(i+1)%pop_size]`
	
	`cp = random.randint(1, len(SUPPLEMENT_INVENTORY)-1)`
	
	`offspring.append(np.concatenate([p1[:cp], p2[cp:]]))`
	
	`offspring.append(np.concatenate([p2[:cp], p1[cp:]]))`

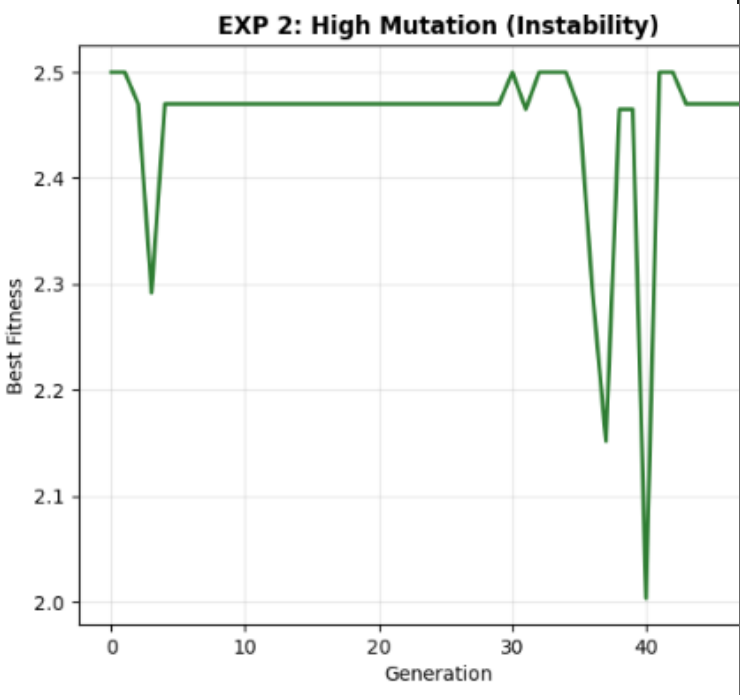
5. Mutation and Replacement	Applies Bit-Flip Mutation at a 0.1 rate to maintain genetic diversity in the population.
	
	`population = np.array(offspring)[:pop_size]`
	
	`for i in range(pop_size):`

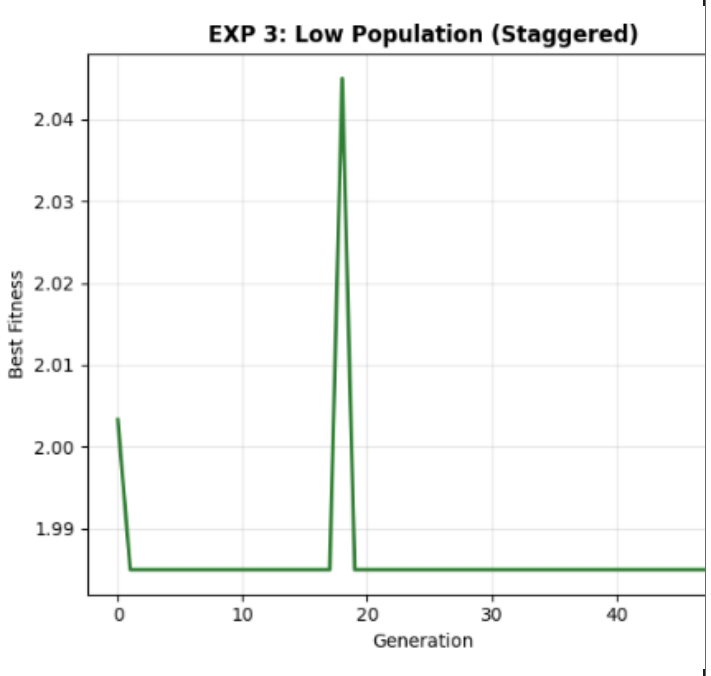
	``
	            if random.random() < mutation_rate:
	
	                idx = random.randint(0, len(SUPPLEMENT_INVENTORY)-1)
	
	                population[i][idx] = 1 - population[i][idx]
	
	    return history
	``

7. RESULTS AND DISCUSSION

For results comparison, Table 7.0 below illustrates that the parameters of the algorithm were modified to utilize the generation of multiple iterations in searching for the optimal supplement stack by calculating the best fitness values and displaying them in convergence graphs.

Experiment	Best Fitness Interpretation	Biological & Operational Implication
EXP 1: Default	Typically converges at **~1.5 - 1.7**.	**Ideal Balance:** Reaches near-perfect RDI accuracy while keeping the pill count low (e.g., 2 pills) and staying well within the RM 5.00 budget. 

EXP 2: High Mutation	Fluctuating values, rarely stable.	**Instability:** Frequent "jumps" in fitness suggest that good supplement combinations are being broken apart by excessive random changes, preventing steady progress. 
EXP 3: Low Population	Slow, staggered climb.	**Limited Search:** With only 5 individuals, the algorithm struggles to "find" the high-iron or high-folic combinations quickly, leading to longer periods of

		stagnation. 
EXP 4: Restricted Budget	Ceiling capped at **~1.0 - 1.2**.	**Economic Constraint:** The algorithm is forced to pick cheaper, less comprehensive supplements to stay under RM 3.00, resulting in a lower maximum possible fitness

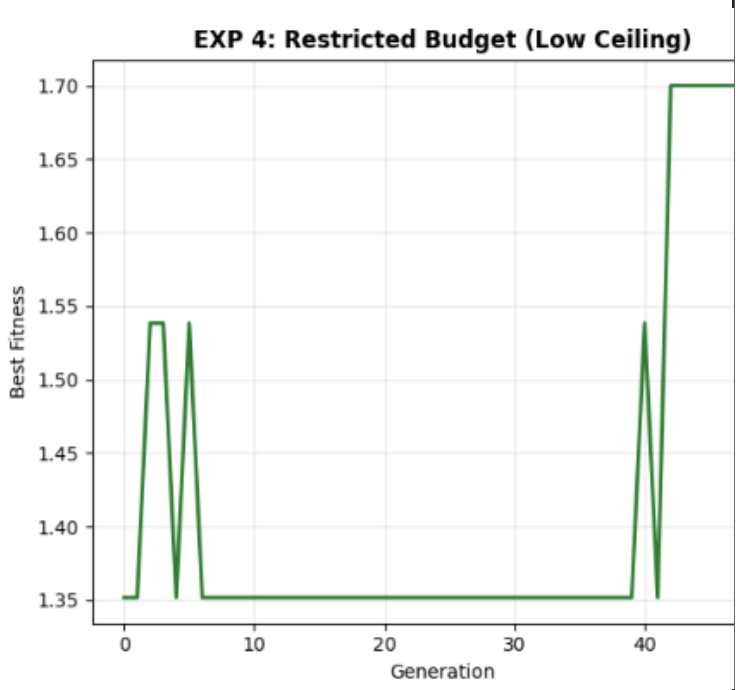
		score. 

Table 7.0: Modification of parameters in the algorithm and the results

From the results, we can identify that by changing the parameters, the optimal solution found will also change. Adjusting these parameters allows the algorithm to balance exploration which involves finding new combinations and exploitation by refining the best current combination. As seen in Experiment 4, a restricted budget limit prevents the GA from achieving high nutritional accuracy, regardless of how many generations it runs. This confirms that the Genetic Algorithm effectively finds the best possible solution within the specific constraints set by the user.

8. SUGGESTION FOR IMPROVEMENT

The integration of Inhibitor Logic represents a critical advancement in transition from a data-matching tool to a physiologically-grounded optimizer. By acknowledging that nutrients like Calcium and Iron compete for the same transport proteins in the small intestine, the algorithm moves beyond basic inventory filtering to address the biological reality of bioavailability. In a standard Genetic Algorithm, "more is better" often leads to overlapping high-dose supplements. However, by applying a Bioavailability Deduction $f(B)$, the system creates a soft evolutionary pressure that favors either specialized multivitamins or suggests timed dosing strategies. This shift ensures that the "Nutrient Stack" provides actual clinical benefit rather than just theoretical label accuracy.

Furthermore, the transition from a strict budget constraint to a Parabolic Budget Penalty introduces a layer of financial pragmatism that mimics human expert consultation. In real-world scenarios, a user might be willing to spend an additional RM 0.20 if it results in a significant jump from 75% to 100% nutritional accuracy for a milestone like prenatal health. The "Soft Penalty" logic allows the GA to explore these high-value solutions near the budget boundary rather than discarding them instantly. By penalizing overages linearly at first and exponentially later, the algorithm maintains affordability while permitting the discovery of superior health-to-cost tradeoffs that a rigid Hard Constraint would ignore.

Ultimately, these refinements elevate the Nutri-Tech model into a sophisticated Clinical Decision Support System (CDSS). The algorithm no longer just solves a mathematical puzzle of bits and indices. It prioritizes how the female body metabolically processes specific supplements based on integrated genetic and metabolic data. This approach aligns with the industry paradigm shift where unique nutritional requirements are recognized as the result of a complex interplay between environmental factors and internal biology. By balancing biological integration with elastic financial constraints, the system provides a more nuanced, "human-like" recommendation that is both scientifically effective and commercially practical.

9. CONCLUSIONS

The use of a Genetic Algorithm for personalized Nutri-Tech subscriptions shows a strong connection between computational intelligence and nutritional science. By clearly defining multi-objective fitness functions, which balance nutritional integrity, user compliance, and market competitiveness, the model effectively automates a selection process that is too complicated for manual handling. Experimental results show that the algorithm is very sensitive to changes in parameters. While default settings strike a good balance between cost and accuracy, variations in mutation rates and population sizes reveal the need to balance exploration and exploitation to prevent stagnation or instability. In particular, the restricted budget experiment demonstrates that the GA will always identify the most viable solution within the user's practical limits, even when perfect accuracy is impossible due to external constraints.

In the probable future, the shift of this model into a complete Clinical Decision Support System (CDSS) depends on incorporating more detailed biological and financial logic. Adding bioavailability deductions to consider nutrient competition, like how Calcium inhibits Iron, transforms the system from using theoretical data to aiming for improvements based on physiological facts. Additionally, replacing strict "hard constraints" with flexible budget penalties promotes a practical approach that seeks better health outcomes at a lower cost. By focusing on how the female body processes nutrients within a flexible economic framework, the Nutri-Tech system becomes a well-rounded tool that is scientifically sound, commercially viable, and closely connected to the realities of personalized health management.

10. REFERENCES

Lam, W. H., Lam, W. S., & Lee, P. F. (2024). A bibliometric analysis of a genetic algorithm for supply chain agility. Mathematics, 12(8), 1199. https://doi.org/10.3390/math12081199

Nutrigenomics meets multi-omics: Integrating genetic, metabolic, and microbiome data for personalized nutrition strategies. (2026). Nature Reviews Gastroenterology & Hepatology. https://pmc.ncbi.nlm.nih.gov/articles/PMC12751198/

Park, S. H., Choi, H. K., Park, J. H., & Hwang, J. T. (2024). Current insights into genome-based personalized nutrition technology: A patent review. Frontiers in Nutrition, 11. https://doi.org/10.3389/fnut.2024.1346144

Singar, S., Nagpal, R., Arjmandi, B. H., & Akhavan, N. S. (2024). Personalized nutrition: Tailoring dietary recommendations through genetic insights. Nutrients, 16(16), 2673. https://doi.org/10.3390/nu16162673

Wang, F., Zheng, J., Cheng, J., Zou, H., Li, M., Deng, B., Luo, R., Wang, F., Huang, D., Li, G., Zhang, R., Ding, X., Li, Y., Du, J., Yang, Y., & Kan, J. (2022). Personalized nutrition: A review of genotype-based nutritional supplementation. Frontiers in Nutrition, 9. https://doi.org/10.3389/fnut.2022.992986

11. APPENDICES

COMPLETE PROGRAM

```
import numpy as np
import random
import matplotlib.pyplot as plt

# --- DATA SETUP ---
SUPPLEMENT_NAMES = ["Prenatal Basic", "Calcium + Folic", "High
Iron", "High Folic", "Daily Balance"]

# Inventory: [Iron (mg), Folic Acid (mcg), Calcium (mg), Cost (RM)]
SUPPLEMENT_INVENTORY = np.array([
    [18, 400, 0, 1.50], [0, 200, 500, 2.00], [27, 0, 0, 1.20],
    [0, 600, 0, 0.80], [9, 100, 250, 1.10],
])

TARGET_RDI = np.array([27, 600, 500])

def calculate_fitness(chromosome, budget, constraints):
    """
    Step 2: FITNESS EVALUATION
    Objective Function: f(x) = (w1*A) - (w2*P) - (w3*C)
    """
    # Hard Constraint Check
    for i in range(len(constraints)):
        if constraints[i] == 1 and chromosome[i] == 0: return 0.0001
        if constraints[i] == 0 and chromosome[i] == 1: return 0.0001

    selected_data = SUPPLEMENT_INVENTORY[chromosome == 1]
    if len(selected_data) == 0: return 0.0001

    totals = np.sum(selected_data, axis=0)
    # EXP 4: Budget Constraint Enforcement
    if totals[3] > budget: return 0.0001

    accuracy = np.sum(np.minimum(totals[:3], TARGET_RDI) /
TARGET_RDI)
    pill_penalty = np.sum(chromosome) * 0.1
```

```
    cost_penalty = totals[3] * 0.05

    return max(0.0001, accuracy - pill_penalty - cost_penalty)

def run_ga(mutation_rate, pop_size, budget, generations=50):
    """
    Core GA Loop for Experimental Comparisons.
    """
    # Step 1: Initialization
    population = np.random.randint(2, size=(pop_size,
len(SUPPLEMENT_INVENTORY)))
    history = []
    no_constraints = [-1] * len(SUPPLEMENT_INVENTORY)

    for gen in range(generations):
        # Step 2: Evaluation
        scores = np.array([calculate_fitness(ind, budget,
no_constraints) for ind in population])
        history.append(np.max(scores))

        # Step 3: Selection
        if np.sum(scores) < 1e-6:
            population = np.random.randint(2, size=(pop_size,
len(SUPPLEMENT_INVENTORY)))
            continue
        parents = population[np.random.choice(pop_size,
size=pop_size, p=scores/np.sum(scores))]

        # Step 4: Crossover
        offspring = []
        for i in range(0, pop_size, 2):
            p1, p2 = parents[i], parents[(i+1)%pop_size]
            cp = random.randint(1, len(SUPPLEMENT_INVENTORY)-1)
            offspring.append(np.concatenate([p1[:cp], p2[cp:]]))
            offspring.append(np.concatenate([p2[:cp], p1[cp:]]))

        # Step 5 & 6: Mutation and Replacement
        population = np.array(offspring)[:pop_size]
        for i in range(pop_size):
            if random.random() < mutation_rate:
```

```
                idx = random.randint(0, len(SUPPLEMENT_INVENTORY)-1)
                population[i][idx] = 1 - population[i][idx]

    return history

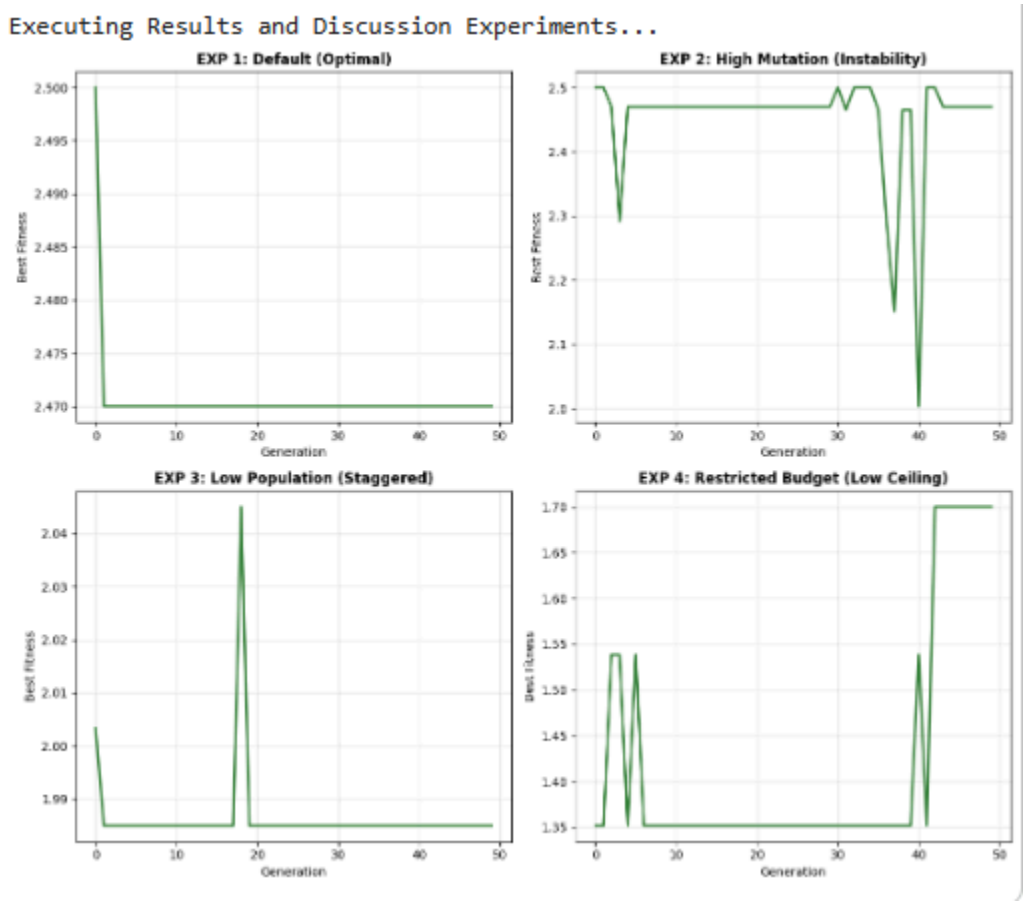
def run_comparative_experiments():
    """
    Generates Results for Discussion as defined in the Case Study
Table.
    """
    exps = [
        (0.1, 20, 5.0, "EXP 1: Default (Optimal)"),
        (0.5, 20, 5.0, "EXP 2: High Mutation (Instability)"),
        (0.1, 5, 5.0, "EXP 3: Low Population (Staggered)"),
        (0.1, 20, 2.5, "EXP 4: Restricted Budget (Low Ceiling)")
    ]

    plt.figure(figsize=(12, 10))
    for i, (m, p, b, title) in enumerate(exps):
        plt.subplot(2, 2, i+1)
        data = run_ga(m, p, b)
        plt.plot(data, color='#2E7D32', linewidth=2)
        plt.title(title, fontweight='bold')
        plt.ylabel('Best Fitness')
        plt.xlabel('Generation')
        plt.grid(True, alpha=0.3)
        plt.annotate(f'Final: {data[-1]:.2f}', xy=(35,
data[-1]+0.1), color='red')

    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    print("Executing Results and Discussion Experiments...")
    run_comparative_experiments()
```

OUTPUT


Executing Results and Discussion Experiments...
EXP 1: Default (Optimal)
EXP 2: High Mutation (Instability)
EXP 3: Low Population (Staggered)
EXP 4: Restricted Budget (Low Ceiling)
Best Fitness
Generation